

Module 9: Introduction to DAX (Data Analysis Expressions)

Overview

DAX (Data Analysis Expressions) is the formula language used in Power BI, Power Pivot, and SQL Server Analysis Services (SSAS) Tabular Models.

DAX allows you to create calculations, business metrics, KPIs, and advanced analytical logic that cannot be achieved through visualizations alone.

Understanding DAX is one of the most important skills for becoming a Power BI Developer, Data Analyst, or BI Analyst.

This module introduces DAX fundamentals, syntax, calculated columns, measures, row context, filter context, and essential business calculations.

Learning Objectives

After completing this module, you will be able to:

- Understand what DAX is
 - Learn DAX syntax
 - Create Calculated Columns
 - Create Measures
 - Understand Row Context
 - Understand Filter Context
 - Write basic DAX formulas
 - Build business calculations
 - Follow DAX best practices
-

Table of Contents

1. What is DAX?
2. Why DAX is Important
3. DAX vs Excel Formulas
4. DAX Syntax
5. Data Types in DAX
6. Operators in DAX
7. Understanding Calculated Columns
8. Understanding Measures
9. Calculated Columns vs Measures
10. Row Context

11. Filter Context
12. Context Transition
13. Common DAX Functions
14. Business Calculations
15. DAX Best Practices
16. Common DAX Mistakes
17. Key Terminologies
18. Practice Exercises
19. Mini Project
20. Module Summary

What is DAX?

Definition

DAX stands for:

Data Analysis Expressions

It is a formula language used to:

- Perform calculations
- Create KPIs
- Build business metrics
- Analyze data dynamically

Where DAX is Used

- Power BI
- Power Pivot
- SSAS Tabular

Examples

Calculate Total Sales:

:contentReference[oaicite:0]{index=0}

In Power BI:

Total Sales =
SUM(Sales[Revenue])

2 Why DAX is Important

Without DAX, Power BI would only display raw data.

DAX enables:

Business Logic

Example:

```
Profit = Revenue - Cost
```

KPIs

Example:

```
Profit Margin %
```

Time Intelligence

Example:

```
Year-to-Date Sales
```

Dynamic Calculations

Results change automatically based on filters.

3 DAX vs Excel Formulas

Many DAX functions resemble Excel formulas.

Similarities

Examples:

```
SUM()  
IF()
```

```
AVERAGE()  
COUNT()
```

Differences

Excel	DAX
Works on Cells	Works on Tables
Static	Dynamic
Single Worksheet	Data Model
Limited Relationships	Relationship Aware

Example

Excel:

```
=A1+B1
```

DAX:

```
Total Revenue =  
SUM(Sales[Revenue])
```



DAX Syntax

Every DAX formula follows a structure.

Measure Syntax

```
Measure Name =  
Function()
```

Example

```
Total Sales =
```

```
SUM(Sales[Revenue])
```

Breakdown

```
Total Sales
```

Measure Name

```
SUM
```

Function

```
Sales[Revenue]
```

Column Reference

5 Data Types in DAX

DAX supports several data types.

Whole Number

```
100
```

Decimal Number

```
100.50
```

Currency

₹5000

Text

Laptop

Date

01-Jan-2026

Boolean

TRUE
FALSE

Operators in DAX

Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
^	Power

Example

```
Profit =  
Sales[Revenue] - Sales[Cost]
```

Comparison Operators

Operator	Meaning
=	Equal
<>	Not Equal
>	Greater Than
<	Less Than
>=	Greater Than Equal
<=	Less Than Equal

Logical Operators

Operator	Meaning
&&	AND
	OR

Understanding Calculated Columns

What is a Calculated Column?

A Calculated Column creates a new column in a table.

The calculation is performed row by row.

Example

Sales Table

Revenue	Cost
1000	700

Create:

```
Profit =  
Sales[Revenue] - Sales[Cost]
```

Result

Revenue	Cost	Profit
1000	700	300

Characteristics

Stored in Model

Consumes memory.

Calculated During Refresh

Not calculated dynamically.

8 Understanding Measures

What is a Measure?

A Measure performs calculations dynamically based on report filters.

Example

```
Total Revenue =  
SUM(Sales[Revenue])
```

Characteristics

Not Stored

Consumes less memory.

Dynamic

Changes according to:

- Filters
 - Slicers
 - User selections
-

9 Calculated Columns vs Measures

Feature	Calculated Column	Measure
Storage	Stored	Not Stored
Calculation Time	Refresh	Query Time
Memory Usage	Higher	Lower
Dynamic	No	Yes
Use in Axis	Yes	No
Use in Values	Yes	Yes

Rule of Thumb

Prefer Measures whenever possible.

10 Row Context

What is Row Context?

Row Context means DAX evaluates one row at a time.

Example

```
Profit =  
Sales[Revenue] - Sales[Cost]
```

Power BI processes:

Row 1

```
1000 - 700 = 300
```

Row 2

```
1500 - 900 = 600
```

This is Row Context.

1 1 Filter Context

What is Filter Context?

Filter Context is created by:

- Visuals
 - Slicers
 - Filters
 - Report Pages
-

Example

A report shows:

Total Revenue

User selects:

Region = North

Measure automatically recalculates.

This behavior is called Filter Context.

1 2 Context Transition

Definition

Context Transition occurs when Row Context becomes Filter Context.

Most commonly triggered by:

CALCULATE()

Why Important?

Foundation of advanced DAX.

1 3 Common DAX Functions

SUM

Adds values.

```
Total Sales =  
SUM(Sales[Revenue])
```

AVERAGE

Calculates average.

```
Average Sales =  
AVERAGE(Sales[Revenue])
```

COUNT

Counts rows.

```
Order Count =  
COUNT(Sales[OrderID])
```

DISTINCTCOUNT

Counts unique values.

```
Customers =  
DISTINCTCOUNT(Sales[CustomerID])
```

MIN

Returns minimum value.

```
Minimum Sales =  
MIN(Sales[Revenue])
```

MAX

Returns maximum value.

```
Maximum Sales =  
MAX(Sales[Revenue])
```

Business Calculations

Total Revenue

```
Total Revenue =  
SUM(Sales[Revenue])
```

Total Cost

```
Total Cost =  
SUM(Sales[Cost])
```

Total Profit

```
Total Profit =  
SUM(Sales[Revenue]) -  
SUM(Sales[Cost])
```

Profit Margin %

```
Profit Margin % =  
DIVIDE(  
    [Total Profit],
```

```
[Total Revenue]  
)
```

Average Order Value

```
Average Order Value =  
DIVIDE(  
    [Total Revenue],  
    [Order Count]  
)
```

1 5 DAX Best Practices

Use Measures Instead of Columns

When possible.

Use Meaningful Names

Example:

```
Total Revenue
```

instead of:

```
TR
```

Use DIVIDE()

Instead of:

```
Revenue / Cost
```

Use:

```
DIVIDE(Revenue, Cost)
```

Organize Measures

Create dedicated measure tables.

Avoid Unnecessary Calculated Columns

They increase memory usage.

16

Common DAX Mistakes

Using Columns Instead of Measures

Leads to inefficient models.

Ignoring Filter Context

Produces incorrect results.

Hardcoding Values

Avoid:

```
Revenue > 1000
```

when values may change.

Poor Naming Conventions

Makes maintenance difficult.

Not Using DIVIDE()

Can cause division errors.



Key Terminologies

Term	Definition
DAX	Data Analysis Expressions
Measure	Dynamic calculation

Term	Definition
Calculated Column	Stored row-by-row calculation
Row Context	Current row being evaluated
Filter Context	Filters affecting calculations
Context Transition	Row Context becoming Filter Context
Function	Built-in DAX operation
Expression	Formula returning a value
KPI	Key Performance Indicator
Aggregation	Summarizing data



Practice Exercises

Exercise 1

Create the following measures:

Total Revenue

Total Cost

Total Profit

Exercise 2

Create a Profit calculated column.

Exercise 3

Calculate:

Average Revenue

Exercise 4

Create:

Distinct Customer Count

Exercise 5

Observe how measures change when slicers are applied.



Mini Project

Sales KPI Dashboard

Dataset

Sales Data containing:

- Revenue
 - Cost
 - CustomerID
 - OrderID
-

Tasks

Create:

Measures

Total Revenue

Total Cost

Total Profit

Profit Margin %

Customer Count

Visuals

- KPI Cards
- Revenue Trend
- Profit Analysis

Objective

Understand how DAX powers business reporting.



Module Summary

In this module, you learned:

- ✓ What DAX Is
- ✓ DAX Syntax
- ✓ Data Types
- ✓ Operators
- ✓ Calculated Columns
- ✓ Measures
- ✓ Row Context
- ✓ Filter Context
- ✓ Context Transition
- ✓ Basic DAX Functions
- ✓ Business Calculations
- ✓ DAX Best Practices

DAX is the analytical engine of Power BI. Mastering DAX enables you to create sophisticated business metrics, KPIs, and advanced analytical solutions.



Next Module

Module 10: Essential DAX Functions

Topics Covered:

- Mathematical Functions
- Statistical Functions
- Logical Functions
- Text Functions
- Date & Time Functions
- Information Functions
- Aggregation Functions
- Real-World Business Calculations
- DAX Function Best Practices